Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 2

Meet the OS You Will Build

Track B: Build Your Own Small OS (xv6)

Elaborated by:
st. gr. FAF-243
Morari Veaceslav

Verified by:
Dumitrascu Marius

Chişinău – 2026

Contents

1 Introduction **2**

2 Environment Setup **2**

3 Building and Booting xv6 **2**

4 Using xv6 **3**
4.1 Observations 3
4.2 Interesting Command Output 4

5 Reading the Source **4**
5.1 Observations 4
5.2 Interesting Command Output 5

6 Writing the First xv6 Program **5**
6.1 Source Code 5
6.2 Adding the Program to the Build 6
6.3 Testing 6
6.4 How the System Call Reaches the Kernel 7

7 Conclusion **7**

1. Introduction

The purpose of this laboratory work is to build a small but real operating system from its source code, boot it, use it, read part of its code, and write a first program that runs inside it.

The operating system used is **xv6**, a teaching OS from MIT. It is a modern re-implementation of Unix Version 6, written in about 9 000 lines of C, and it runs on the RISC-V architecture under the QEMU emulator. In Laboratory Work No. 1 the operating system was observed only from the outside; xv6 is small enough that both sides can be read: the user program that asks for a service and the kernel code that answers it.

2. Environment Setup

The work was performed in Ubuntu 26.04 under WSL2 instead of the course virtual machine, so the toolchain had to be installed first:

```
sudo apt update
sudo apt install -y git build-essential gdb-multiarch qemu-system-misc \
    gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
sudo apt install -y qemu-system-riscv
```

The cross-compiler `riscv64-linux-gnu-gcc` runs on an x86 processor but produces RISC-V machine code, and `qemu-system-riscv64` emulates a complete RISC-V computer. In recent Ubuntu versions the RISC-V emulator is no longer included in `qemu-system-misc`, so the package `qemu-system-riscv` had to be installed separately. The source code was then cloned from the official repository:

```
git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv
ls
```

```
LICENSE  Makefile  README  kernel  mkfs  test-xv6.py  user
```

3. Building and Booting xv6

The kernel and the user programs were compiled, and xv6 was launched in the emulator with a single command:

```
make qemu
```

This command compiles every file in `kernel/` into the kernel image `kernel/kernel`, compiles every program in `user/`, writes the programs into the disk image `fs.img`, and starts QEMU. After a few seconds the kernel came up and started the shell:

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp
    3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=
   none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-
   bus.0
```

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

The emulated machine has 128 MB of RAM (`-m 128M`), three processor cores (`-smp 3`), which RISC-V calls *harts*, and one virtual disk (`fs.img`). As in Linux, the kernel creates the first user process `init`, which then starts the shell. To leave xv6, the QEMU sequence `Ctrl-A`, then `X` is used.

4. Using xv6

At the xv6 `$` prompt, the built-in programs were run:

```
$ ls
$ cat README
$ echo hello xv6
$ ls | grep c
$ wc README
```

4.1. Observations

1. Three programs that xv6 ships with are `cat`, `grep` and `wc`. Other examples are `ls`, `echo`, `mkdir`, `rm`, `kill` and the shell `sh`.

2. For the pipe `ls | grep c` to work, two OS features must exist:

   - **processes** – the shell uses `fork` and `exec` to create two processes, `ls` and `grep`, that run at the same time;
   - **inter-process communication** – the kernel creates a pipe, a buffer inside the kernel with a write end and a read end. `ls` writes into one end and `grep` reads from the other. Because processes cannot touch each other's memory, the kernel moves the data between them.

3. The xv6 shell works like the Linux shell from Laboratory Work No. 1, with the same kind of commands, pipes and redirection, but it is much simpler: it has no command history, no tab completion, no variables, and files have no owners or permissions.

4.2. Interesting Command Output

The pipe and the `wc` command produced the following result:

```
$ ls | grep c
cat            2 3 36728
echo           2 4 35592
wc             2 17 37728
sync           2 22 34944
console        3 23 0
$ wc README
48 336 2441 README
```

The columns printed by `ls` are the name, the file type, the inode number and the size in bytes. Type 1 is a directory, type 2 a regular file and type 3 a device. The entry `console` has type 3, which shows that even in xv6 the terminal is presented as a file, the same "everything is a file" idea that was seen in `/dev` on Linux.

5. Reading the Source

After quitting QEMU, the source tree was examined. The whole OS is split into two folders: `kernel/`, the operating system itself, and `user/`, the ordinary programs that run on top of it. A user program and the list of system calls were read, and the kernel implementation of `read` and `write` was located:

```
sed -n "1,40p" user/ls.c
sed -n "1,30p" user/cat.c
cat user/user.h
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
```

The core of `user/cat.c` is the following loop:

```
while ((n = read(fd, buf, sizeof(buf))) > 0) {
  if (write(1, buf, n) != n) {
    fprintf(2, "cat: write error\n");
    exit(1);
  }
}
```

5.1. Observations

1. `user/cat.c` uses the following system calls:
    - `open` – asks the kernel to open a file and return a file descriptor;
    - `read` – asks the kernel to copy up to n bytes from the file into the program's buffer;
    - `write` – asks the kernel to write the bytes to file descriptor 1, the standard output (the screen);
    - `close` – asks the kernel to close the file;

- `exit` – asks the kernel to terminate the process.

`fprintf(2, ...)` is not a system call itself; it is a library function that calls `write` on file descriptor 2, the standard error.

2. `sys_read` is implemented in the file `kernel/sysfile.c`, on **line 69** (`sys_write` is on line 83).

3. The code in `kernel/` is the operating system itself and runs in privileged (supervisor) mode with direct access to the hardware, while the code in `user/` consists of ordinary programs without privileges that can only ask the kernel for services through system calls.

5.2. Interesting Command Output

```
$ grep -n "sys_read\|sys_write" kernel/sysfile.c
69:sys_read(void)
83:sys_write(void)
```

While reading `user/user.h`, I noticed that the list of system calls contains no `sleep`, but has `pause` instead:

```
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
```

In the current version of xv6 the system call was renamed from `sleep` to `pause` (upstream commit `f5dea58`: "user sleep() -> pause()"). This matters for the next part, because the program from the lab sheet calls `sleep()` and does not compile on this version:

```
error: implicit declaration of function 'sleep'
```

6. Writing the First xv6 Program

A new user program, `sleep`, was added. It pauses for a number of clock ticks (one tick is one timer interrupt, about 0.1 s in xv6) by calling a system call that already exists in the kernel.

6.1. Source Code

The file `user/sleep.c` was created. Because the name of the system call differs between versions of xv6, the program includes `kernel/syscall.h`, the list of system call numbers, and chooses the right call at compile time: if `SYS_pause` exists it calls `pause()`, otherwise the older `sleep()`. In this way the same file builds on both the new and the old xv6.

```
#include "kernel/types.h"
#include "kernel/syscall.h"
```

```
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
#ifdef SYS_pause
  pause(atoi(argv[1]));   // system call into the kernel (new xv6 name)
#else
  sleep(atoi(argv[1]));   // system call into the kernel (older xv6 name)
#endif
  exit(0);
}
```

6.2. Adding the Program to the Build

The new program was added to the `UPROGS` list in the `Makefile`, right after `_echo`, indented with a tab like the other lines:

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        $U/_sleep\
        $U/_forktest\
        ...
```

Without this line the file would be ignored by the build and the program would not appear on the xv6 disk.

6.3. Testing

xv6 was rebuilt with `make qemu`, and the program was tested inside it:

```
$ ls
...
sleep          2 5 35280
...
$ sleep
usage: sleep <ticks>
$ sleep 10
$ sleep 100
$
```

The new file `sleep` appeared on the disk. Without an argument the program printed its usage message, and `sleep 10` and `sleep 100` paused for about 1 and 10 seconds before the prompt came back.

6.4. How the System Call Reaches the Kernel

When `sleep 10` runs, the call `pause(10)` goes through the following steps:

1. `user/usys.S` puts the number of the system call (`SYS_pause` = 13) into register `a7` and executes the instruction `ecall`, which switches the processor into supervisor mode and enters the kernel.

2. `syscall()` in `kernel/syscall.c` reads `a7` and calls the handler from its table.

3. `sys_pause()` in `kernel/sysproc.c` (line 68) remembers the current tick count and puts the process to sleep.

4. On every timer interrupt, `kernel/trap.c` increments `ticks` and wakes up sleeping processes; after 10 ticks `sys_pause` returns and the program calls `exit(0)`.

7. Conclusion

In this laboratory work the teaching operating system **xv6** was compiled from source with a RISC-V cross-compiler, booted in QEMU and used as a small Unix system. Its programs, including a pipe, behaved like their Linux counterparts, which shows that xv6 keeps the main ideas of Unix: processes, file descriptors, pipes and "everything is a file".

Reading the source showed the central organisation of an operating system: the privileged kernel in `kernel/` controls the hardware, and the unprivileged programs in `user/` can only ask it for services through system calls such as `read` and `write`.

Finally, a new program, `sleep`, was written, added to the `Makefile` and tested successfully. It makes a real system call into a kernel that I compiled myself. During the work it was found that the current xv6 renamed this system call to `pause`; the program handles both names, so it works on old and new versions of xv6.